Agentic AI Architecture

Speaker Notes

Day 2 — 27 October 2026 — 40 minutes

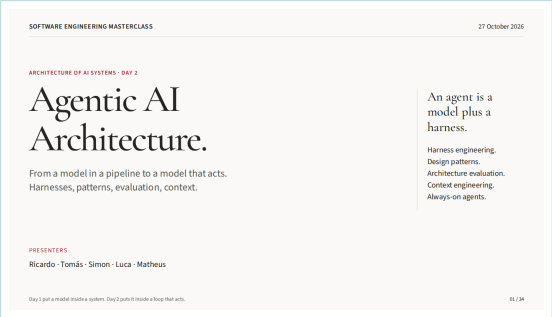
Each slide has its current image, delivery cues, the key points shown on the slide, and a spoken script in short paragraphs. Read it or adapt it in your own voice. Optional preparation boxes explain terms, give an example to keep in reserve, and flag limits of the evidence; they are not additional text to read aloud. Examples marked optional are teaching illustrations, not reported experiments. There is no running example: each topic stands on its own paper, and real systems — Pi, Pi Durable and Hermes — appear as evidence. The right-hand column of each slide is read by the audience; the directions for you are in the CUES box here.

Time	Presenter	Slides
0–2 min	All	1–2
2–9 min	Ricardo	3–8
9–15 min	Tomás	9–14
15–21 min	Simon	15–19
21–29 min	Luca	20–26
29–37 min	Matheus	27–33
37–40 min	All	34 (discussion)

Keys: Right/Space reveals the next diagram part, then the next slide; Left reverses; G opens the contents; F fullscreen.

01 / 34 Agentic AI Architecture

Opening • All presenters



CUES

- Timing: about 60 seconds spoken; preparation is optional.

SPOKEN SCRIPT — short paragraphs for delivery

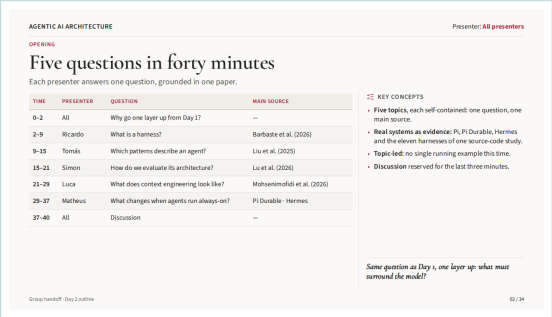
- Welcome back. Last week we asked how you build a system around a machine-learning model that behaves statistically. Today we ask the same question one layer up: what do you build around a model that does not just predict, but acts?
- This time there is no single running example. Each of the five topics stands on its own paper. Where it helps, we show real systems as evidence: Pi, a minimal open-source coding agent; Pi Durable, a library for agents that survive crashes; and Hermes, an always-on agent.
- Five of us will each answer one question. Then we keep a few minutes for discussion.

OPTIONAL PREPARATION — example or clarification

A foundation model is the pretrained model supplying language and reasoning capabilities. An agent adds a loop that can choose and execute actions. Optional illustration: a chatbot describes a file change; a coding agent can edit the file and run a test. Neither label guarantees that the result is correct.

02 / 34 Five questions in forty minutes

Opening • All presenters



CUES

- Static figure or table: no builds. If it is a paper figure, name the source aloud first.
- Do: Read only the question column aloud; the times are for us.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Each presenter answers one question, grounded in one paper.
- **Five topics**, each self-contained: one question, one main source.
- **Real systems as evidence:** Pi, Pi Durable, Hermes and the eleven harnesses of one source-code study.
- **Topic-led:** no single running example this time.
- **Discussion** reserved for the last three minutes.
- *Same question as Day 1, one layer up: what must surround the model?*

SPOKEN SCRIPT — short paragraphs for delivery

- Here is the plan for the next forty minutes. Each topic is self-contained and grounded in one main source, so you can follow each part on its own.
- Ricardo starts with the harness: what sits around the model. Tomás gives us a vocabulary of design patterns. Simon asks how we evaluate an agent’s architecture. Luca looks at context engineering in real projects. Matheus closes with what changes when agents run all the time.
- The last column tells you where to read more after today.

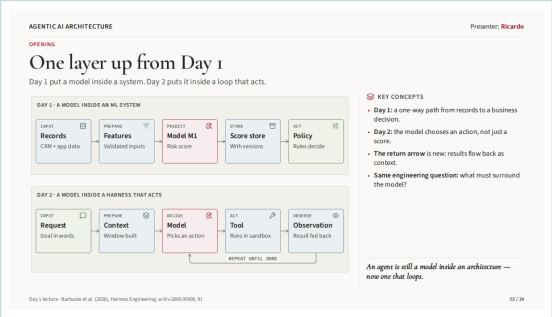
OPTIONAL PREPARATION — example or clarification

The five questions connect design to evidence: identify the harness, describe its patterns, evaluate required qualities, inspect the information available to it, then consider unattended operation. Each topic has its own source; the recurring systems are comparison points, not a running case study.

Reading anchor: Group handoff
Day 2 outline

03 / 34 One layer up from Day 1

Opening • Ricardo



CUES

- Handover: **Ricardo** takes over here.
- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Point at the top lane first, then the bottom lane; finish on the return arrow.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Day 1 put a model inside a system. Day 2 puts it inside a loop that acts.
- **Day 1:** a one-way path from records to a business decision.
- **Day 2:** the model chooses an action, not just a score.
- **The return arrow** is new: results flow back as context.
- **Same engineering question:** what must surround the model?
- *An agent is still a model inside an architecture — now one that loops.*

SPOKEN SCRIPT — short paragraphs for delivery

- Let me connect this to Day 1. In the top lane you see last week’s architecture: records become features, the model produces a risk score, the score is stored, and a separate business policy decides what happens. Information flows one way.
- In the bottom lane the model is no longer the last step before a decision. It reads a request, picks an action, a tool runs, and the result comes back. That return arrow is the new part. The output of one step becomes the input of the next.
- So the engineering question stays the same — what must surround the model? — but the answer now has to manage a loop.

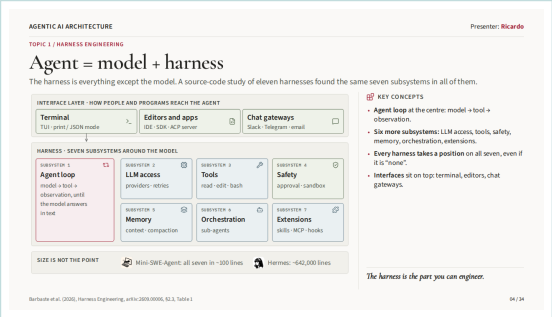
OPTIONAL PREPARATION — example or clarification

Optional illustration: a conventional model returns a churn score once; an agent may inspect a failed test, read the relevant file, edit it and rerun the test. The return path carries new observations into a later decision. A loop adds state and failure handling, even when each individual tool is deterministic.

Reading anchor: Day 1 lecture
Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §1

04 / 34 Agent = model + harness

Topic 1 / Harness engineering • Ricardo



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Start at the agent loop, then sweep across the six boxes; end on the size line at the bottom.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The harness is everything except the model. A source-code study of eleven harnesses found the same seven subsystems in all of them.
- **Agent loop** at the centre: model → tool → observation.
- **Six more subsystems:** LLM access, tools, safety, memory, orchestration, extensions.
- **Every harness takes a position** on all seven, even if it is “none”.
- **Interfaces** sit on top: terminal, editors, chat gateways.
- *The harness is the part you can engineer.*

SPOKEN SCRIPT — short paragraphs for delivery

- A simple equation from this year’s literature: an agent is a model plus a harness. The harness is everything except the model.
- A source-code study of eleven production coding agents found the same seven subsystems in all of them. In the centre is the agent loop. Around it: access to the language model, tools, safety, memory and context, orchestration of sub-agents, and extensions. On top sits the interface layer: a terminal, an editor, or a chat app.
- The size varies enormously. Mini-SWE-Agent covers all seven in about a hundred lines; Hermes needs around six hundred and forty thousand. But every harness takes a position on all seven, even when the position is “we don’t do this”.

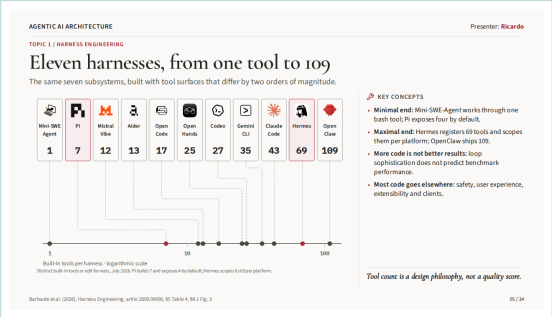
OPTIONAL PREPARATION — example or clarification

If asked what belongs to the harness: the model supplies a proposed tool call; the harness parses it, checks permissions, executes the tool and records the result. Orchestration means coordinating work between agents or tasks. A subsystem can be intentionally minimal; its presence in a taxonomy does not imply a large implementation.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §2.3, Table 1

05 / 34 Eleven harnesses, from one tool to 109

Topic 1 / Harness engineering • Ricardo



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Reveal left to right: start at the single bash tool, end at Hermes and OpenClaw.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The same seven subsystems, built with tool surfaces that differ by two orders of magnitude.
- **Minimal end:** Mini-SWE-Agent works through one bash tool; Pi exposes four by default.
- **Maximal end:** Hermes registers 69 tools and scopes them per platform; OpenClaw ships 109.
- **More code is not better results:** loop sophistication does not predict benchmark performance.
- **Most code goes elsewhere:** safety, user experience, extensibility and clients.
- *Tool count is a design philosophy, not a quality score.*

SPOKEN SCRIPT — short paragraphs for delivery

- All eleven harnesses have the same seven subsystems, but look at how many tools they ship. The axis is logarithmic.
- At the minimal end, Mini-SWE-Agent does everything through a single bash tool. Pi builds seven tools and exposes only four by default; everything else is an extension. In the middle sit Mistral Vibe, Aider, OpenCode, OpenHands, Codex, Gemini CLI and Claude Code, from twelve to forty-three.
- At the maximal end, Hermes registers sixty-nine tools and shows each platform only a subset; OpenClaw ships more than a hundred, including chat channels and media. The study finds that loop sophistication does not predict benchmark results; most of the code goes into safety, user experience, extensibility and clients. Tool count is a design philosophy, not a quality score.

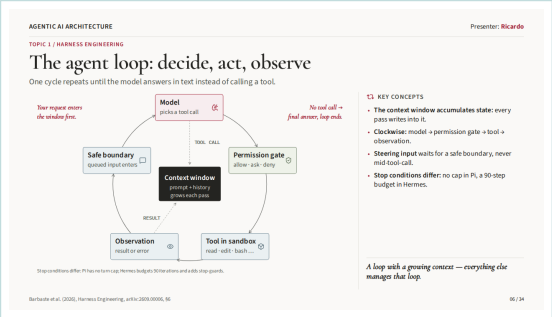
OPTIONAL PREPARATION — example or clarification

Distinguish registered tools from tools currently exposed to the model. Optional illustration: a deployment might register a shell and a deployment tool but expose only the shell in a review session. A larger tool catalogue increases choice and configuration work; these counts alone do not establish capability or safety.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §5 Table 4, §8.1 Fig. 3

06 / 34 The agent loop: decide, act, observe

Topic 1 / Harness engineering • Ricardo



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Walk the ring clockwise from the model; point at the hub each time something is written back.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- One cycle repeats until the model answers in text instead of calling a tool.
- **The context window accumulates state:** every pass writes into it.
- **Clockwise:** model → permission gate → tool → observation.
- **Steering input** waits for a safe boundary, never mid-tool-call.
- **Stop conditions differ:** no cap in Pi, a 90-step budget in Hermes.
- *A loop with a growing context — everything else manages that loop.*

SPOKEN SCRIPT — short paragraphs for delivery

- This is the heart of every agent. In the middle is the context window: the prompt plus everything that has happened so far. It grows with every pass.
- Walk clockwise. The model reads the window and picks a tool call. A permission gate decides whether it may run. The tool runs, and its result — or its error — is written back into the window. If a person wants to steer, their message waits for a safe boundary, never in the middle of a tool call.
- The loop ends when the model answers in text instead of calling a tool. Harnesses disagree on whether to add a safety net: Pi has no turn cap at all, while Hermes gives each turn a budget of ninety iterations and adds guards that stop a premature final answer.

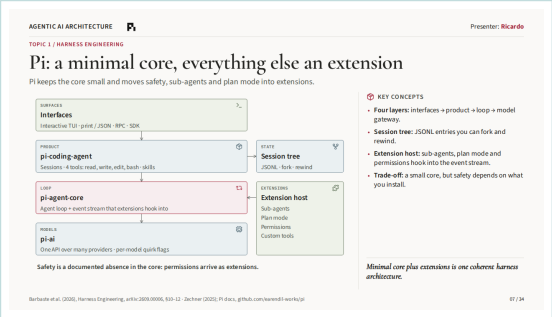
OPTIONAL PREPARATION — example or clarification

Optional illustration: the model proposes running tests; the gate allows it; the test tool returns an error; the next model call sees that error and chooses what to inspect. An iteration cap limits repeated calls, but also consider time and cost. A text answer terminates this loop; it does not prove the goal was achieved.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §6

07 / 34 Pi: a minimal core, everything else an extension

Topic 1 / Harness engineering • Ricardo



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Read the four layers top to bottom, then point at the extension host.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Pi keeps the core small and moves safety, sub-agents and plan mode into extensions.
- **Four layers:** interfaces → product → loop → model gateway.
- **Session tree:** JSONL entries you can fork and rewind.
- **Extension host:** sub-agents, plan mode and permissions hook into the event stream.
- **Trade-off:** a small core, but safety depends on what you install.
- *Minimal core plus extensions is one coherent harness architecture.*

SPOKEN SCRIPT — short paragraphs for delivery

- Pi is the most deliberate example of a small harness. Read it top to bottom. Interfaces at the top. Then the coding-agent product with sessions and four default tools: read, write, edit and bash. Then the agent loop. At the bottom, one API over many model providers.
- Two things sit on the side. Sessions are stored as a tree of JSON lines, so you can fork a conversation or go back to an earlier point. And an extension host: sub-agents, plan mode and even permissions are extensions, not part of the core.
- That is a real trade-off. The core stays small and easy to understand, but safety depends on what you install. The harness study calls Pi’s loop the purest in the corpus after Mini-SWE-Agent.

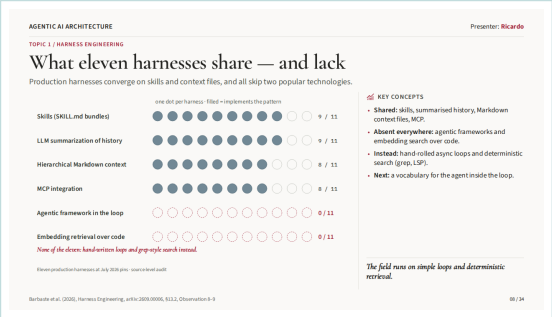
OPTIONAL PREPARATION — example or clarification

JSONL means JSON Lines: each line is a structured record. A session tree records branches in the conversation so a user can return to a prior point. Optional illustration: try two repair approaches on different branches. Extension-based permissions require checking which extensions and restrictions are actually active.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §10–12
Zechner (2025); Pi docs, github.com/earendil-works/pi

08 / 34 What eleven harnesses share — and lack

Topic 1 / Harness engineering • Ricardo



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Pause on the two empty rows of dots. Then hand over to Tomás.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Production harnesses converge on skills and context files, and all skip two popular technologies.
- **Shared:** skills, summarised history, Markdown context files, MCP.
- **Absent everywhere:** agentic frameworks and embedding search over code.
- **Instead:** hand-rolled async loops and deterministic search (grep, LSP).
- **Next:** a vocabulary for the agent inside the loop.
- *The field runs on simple loops and deterministic retrieval.*

SPOKEN SCRIPT — short paragraphs for delivery

- What do eleven production harnesses have in common? Skills and summarised history appear in nine of them; hierarchical Markdown context files and the Model Context Protocol, or MCP, in eight. MCP is a standard way to expose tools and resources to an agent.
- The surprising part is the two empty rows. None of the eleven builds its loop on an agentic framework, and none uses embedding search over code. They write their own simple loops and search the code with deterministic tools like grep and the language server.
- So the field is converging on simple loops and deterministic retrieval. Tomás will now give us a vocabulary to describe what happens inside such an agent.

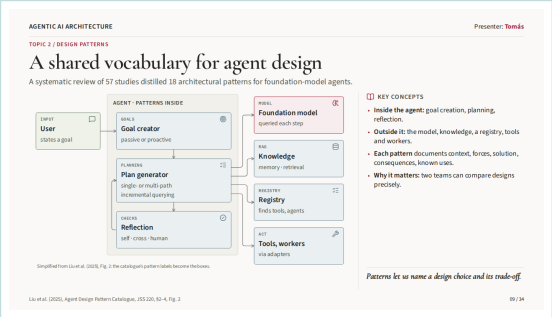
OPTIONAL PREPARATION — example or clarification

MCP means Model Context Protocol. Deterministic retrieval here means explicit code-search and navigation tools, such as text search or finding a symbol definition, rather than embedding similarity search. These findings concern the eleven systems and versions studied; avoid generalising them to every agent or to all forms of retrieval.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §13.2, Observation 8–9

09 / 34 A shared vocabulary for agent design

Topic 2 / Design patterns • Tomás



CUES

- Handover: **Tomás** takes over here.
- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Follow the user's goal from left to right before naming any pattern.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- A systematic review of 57 studies distilled 18 architectural patterns for foundation-model agents.
- **Inside the agent:** goal creation, planning, reflection.
- **Outside it:** the model, knowledge, a registry, tools and workers.
- **Each pattern** documents context, forces, solution, consequences, known uses.
- **Why it matters:** two teams can compare designs precisely.
- *Patterns let us name a design choice and its trade-off.*

SPOKEN SCRIPT — short paragraphs for delivery

- Thank you, Ricardo. A systematic review of fifty-seven studies distilled eighteen architectural patterns for agents built on foundation models.
- This diagram is our simplified version of the catalogue's overview. A user states a goal. Inside the agent, a goal creator turns it into a goal, a plan generator breaks it down, and reflection checks the work. Outside the agent are the foundation model, a knowledge store, a registry of capabilities, and the tools and worker agents reached through adapters.
- Each pattern is documented with its context, forces, solution, consequences and known uses. That gives two teams a precise way to compare designs.

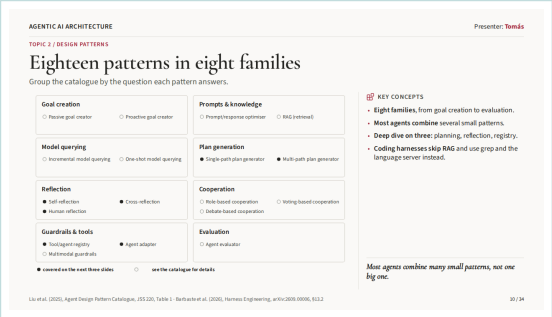
OPTIONAL PREPARATION — example or clarification

A pattern describes a recurring problem, the forces affecting a choice, and a reusable solution with consequences. Optional illustration: the same planning pattern can be implemented by two different libraries. A pattern catalogue is a vocabulary for comparing designs, not a prescription to implement every pattern.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §2–4, Fig. 2

10 / 34 Eighteen patterns in eight families

Topic 2 / Design patterns • Tomás



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Name the eight family titles only; point at the filled dots for the next three slides.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Group the catalogue by the question each pattern answers.
- **Eight families**, from goal creation to evaluation.
- **Most agents combine** several small patterns.
- **Deep dive on three:** planning, reflection, registry.
- **Coding harnesses skip RAG** and use grep and the language server instead.
- *Most agents combine many small patterns, not one big one.*

SPOKEN SCRIPT — short paragraphs for delivery

- The eighteen patterns fall into eight families, each answering one design question: how goals are created, how prompts and knowledge are handled, how often the model is queried, how plans are generated, who reflects, how agents cooperate, how tools and guardrails are managed, and how the agent is evaluated.
- Most real agents combine many of these small patterns rather than following one big one. One interesting gap: coding harnesses skip retrieval-augmented generation over code and explore with grep and the language server instead.
- We go deeper into three families: planning, reflection and registries.

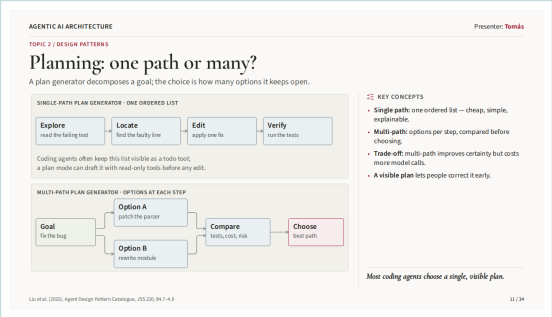
OPTIONAL PREPARATION — example or clarification

If the list feels abstract, separate two questions: planning chooses what to do; reflection assesses an output or plan. Tool selection chooses a capability, while a registry describes the capabilities available. The coding-agent retrieval observation comes from the harness study, not the pattern catalogue.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, Table 1
Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §13.2

11 / 34 Planning: one path or many?

Topic 2 / Design patterns • Tomás



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Top lane first, then the branching lane; ask which one a coding agent should use.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- A plan generator decomposes a goal; the choice is how many options it keeps open.
- **Single path:** one ordered list — cheap, simple, explainable.
- **Multi-path:** options per step, compared before choosing.
- **Trade-off:** multi-path improves certainty but costs more model calls.
- **A visible plan** lets people correct it early.
- *Most coding agents choose a single, visible plan.*

SPOKEN SCRIPT — short paragraphs for delivery

- A plan generator breaks a goal into steps. The design choice is how many options it keeps open.
- The top lane is a single-path plan: one ordered list — explore, locate, edit, verify. It is cheap, simple and easy to explain. The bottom lane is multi-path: at each step the agent keeps options, compares them and then chooses. That improves certainty but costs more model calls.
- Most coding agents choose a single plan and keep it visible, for example as a todo list. A visible plan lets a person correct the direction early.

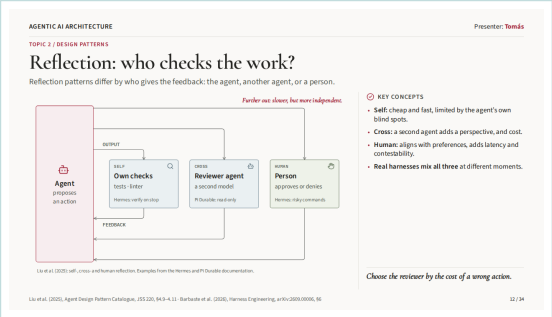
OPTIONAL PREPARATION — example or clarification

Optional illustration: a single-path planner tries a small patch and tests it; a multi-path planner considers a patch, a dependency change and a rewrite before selecting one. More alternatives can increase exploration and cost without guaranteeing a better result. A visible checklist is a coordination aid, not evidence that each step was completed.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §4.7–4.8

12 / 34 Reflection: who checks the work?

Topic 2 / Design patterns • Tomás



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: One loop at a time, from the inside out.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Reflection patterns differ by who gives the feedback: the agent, another agent, or a person.
- **Self:** cheap and fast, limited by the agent's own blind spots.
- **Cross:** a second agent adds a perspective, and cost.
- **Human:** aligns with preferences, adds latency and contestability.
- **Real harnesses mix all three** at different moments.
- *Choose the reviewer by the cost of a wrong action.*

SPOKEN SCRIPT — short paragraphs for delivery

- Reflection means checking the agent's own output. The patterns differ by who gives the feedback.
- Picture three loops around one agent. In the innermost loop, self-reflection, the agent checks itself, for example with tests and a linter. Hermes uses a version of this: it refuses a final answer if code changed without fresh test evidence. In the middle loop, cross-reflection, a second agent reviews the work; Pi Durable's documentation suggests a reviewer with a cheaper model and read-only tools. In the outer loop, human reflection, a person approves or denies, like Hermes's approval step before dangerous commands.
- Moving outward can add independence, but usually adds coordination or waiting time. Self-reflection is cheap but shares the agent's blind spots. Humans add latency but also contestability. Choose the reviewer by the cost of a wrong action.

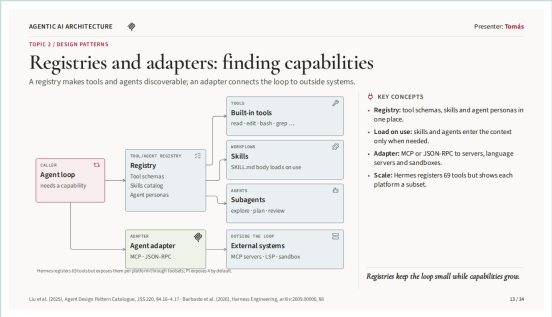
OPTIONAL PREPARATION — example or clarification

Optional illustration: self-review finds a failed test; a second agent questions the assumptions of the fix; a person decides whether the change is acceptable. Cross-review is only partly independent when reviewers share the same model or evidence. Tests provide external feedback, but passing tests cover only the behaviours they check.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §4.9–4.11
Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §6

13 / 34 Registries and adapters: finding capabilities

Topic 2 / Design patterns • Tomás



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Follow the loop → registry → capability path, then the adapter path below.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- A registry makes tools and agents discoverable; an adapter connects the loop to outside systems.
- **Registry:** tool schemas, skills and agent personas in one place.
- **Load on use:** skills and agents enter the context only when needed.
- **Adapter:** MCP or JSON-RPC to servers, language servers and sandboxes.
- **Scale:** Hermes registers 69 tools but shows each platform a subset.
- *Registries keep the loop small while capabilities grow.*

SPOKEN SCRIPT — short paragraphs for delivery

- As capabilities grow, the agent needs a way to find them. A registry holds tool schemas, a catalog of skills and agent personas in one place, and the loop asks the registry instead of hard-coding everything.
- Skills and sub-agents enter the context only when they are needed. Below, an adapter translates between the loop and an external interface. MCP means Model Context Protocol: it exposes tools and resources in a standard form. A language server supplies code information, while a sandbox provides an execution boundary.
- Scale makes this necessary. Hermes registers sixty-nine tools but shows each platform only a subset; Pi exposes four by default. Registries keep the loop small while the capabilities grow.

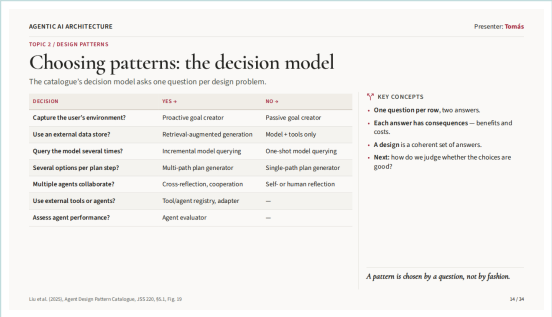
OPTIONAL PREPARATION — example or clarification

Optional illustration: a registry lists a search tool and its input schema; an adapter turns a selected call into the external service request and converts its result back. A schema describes valid inputs, not permission to act. Distinguish discovering a capability from authorising and safely executing it.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §4.16–4.17
Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §8

14 / 34 Choosing patterns: the decision model

Topic 2 / Design patterns • Tomás



CUES

- Static figure or table: no builds. If it is a paper figure, name the source aloud first.
- Do: Read one row aloud as an example. Then hand over to Simon.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The catalogue's decision model asks one question per design problem.
- **One question per row**, two answers.
- **Each answer has consequences** — benefits and costs.
- **A design** is a coherent set of answers.
- **Next:** how do we judge whether the choices are good?
- *A pattern is chosen by a question, not by fashion.*

SPOKEN SCRIPT — short paragraphs for delivery

- The catalogue also offers a decision model: one question per design problem. Take the second row. Do you use an external data store? Retrieval-augmented generation, or RAG, retrieves relevant external material and supplies it to the model before an answer. An external store makes that pattern relevant; it does not mean every database access must use RAG.
- Every answer has consequences, benefits and costs, which the catalogue documents. A good design is a coherent set of answers, not a collection of fashionable patterns.
- But how do we know whether the choices are good? Simon takes that question.

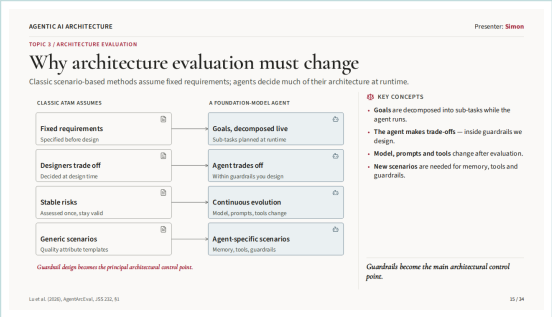
OPTIONAL PREPARATION — example or clarification

RAG means retrieval-augmented generation. Optional illustration: retrieve a current policy passage and include it with the question before generating an answer. The passage may still be irrelevant or stale. Use the decision model to expose consequences and assumptions, rather than treating each yes/no answer as an automatic architecture decision.

Reading anchor: Liu et al. (2025), Agent Design Pattern Catalogue, JSS 220, §5.1, Fig. 19

15 / 34 Why architecture evaluation must change

Topic 3 / Architecture evaluation • Simon



CUES

- Handover: **Simon** takes over here.
- Build: press Right 4 times to reveal the complete figure; once more to advance.
- Do: Read each row left to right: classic assumption, then what an agent does instead.
- Timing: about 60 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Classic scenario-based methods assume fixed requirements; agents decide much of their architecture at runtime.
- **Goals** are decomposed into sub-tasks while the agent runs.
- **The agent makes trade-offs** — inside guardrails we design.
- **Model, prompts and tools** change after evaluation.
- **New scenarios** are needed for memory, tools and guardrails.
- *Guardrails become the main architectural control point.*

SPOKEN SCRIPT — short paragraphs for delivery

- Thank you, Tomás. ATAM means Architecture Tradeoff Analysis Method. It brings stakeholders together to evaluate an architecture against quality scenarios: for example, whether a service can recover quickly without compromising data integrity. The result is an analysis of risks and trade-offs, rather than one overall score.
- AgentArcEval adapts this approach to foundation-model-based agents. A broad user goal becomes concrete tasks while the agent runs. The agent may choose tools and trade speed against additional checking. Architects still control the boundaries: permissions, budgets, approval gates and fallback behaviour.
- The model, instructions and tools can also change after the review. So we must examine the surrounding components and their interactions, then revisit our assumptions using operational evidence. Traditional evaluation remains useful; agents need additional scenarios and continuing review.

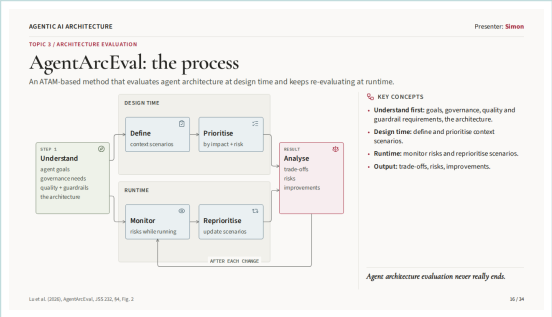
OPTIONAL PREPARATION — example or clarification

ATAM background: SEI, `https://www.sei.cmu.edu/library/architecture-tradeoff-analysis-method-collection/`. Quality attributes include latency, availability and security. Optional illustration: extra validation may improve safety while increasing latency. AgentArcEval extends scenario-based analysis; do not claim that ATAM assumes nothing ever changes or that all agent architecture is generated at runtime.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232, §1

16 / 34 AgentArcEval: the process

Topic 3 / Architecture evaluation • Simon



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Start at Understand; then the upper path, then the lower path and the loop back.
- Timing: about 80 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- An ATAM-based method that evaluates agent architecture at design time and keeps re-evaluating at runtime.
- **Understand first:** goals, governance, quality and guardrail requirements, the architecture.
- **Design time:** define and prioritise context scenarios.
- **Runtime:** monitor risks and reprioritise scenarios.
- **Output:** trade-offs, risks, improvements.
- *Agent architecture evaluation never really ends.*

SPOKEN SCRIPT — short paragraphs for delivery

- AgentArcEval begins with the people responsible for the system. Clarify the agent's goals and users, review governance obligations, agree quality requirements and guardrails, and inspect the architecture.
- At design time, turn a general concern into a concrete scenario. For example, a tax professional asks a question and needs an answer with correct legal references. Set the response measure, prioritise the scenario, and walk through the relevant components. Does retrieval supply current sources? Can the generator cite them? Record gaps, trade-offs and recommended changes.
- At runtime, collect evidence from execution traces, evaluation results and user feedback. If references become outdated after a data or model change, revisit that scenario, adjust its priority, and analyse whether retrieval, validation or another component must change. Then evaluate the improvement.
- This is a review method supported by monitoring; it is not an automatic tool that certifies an architecture.

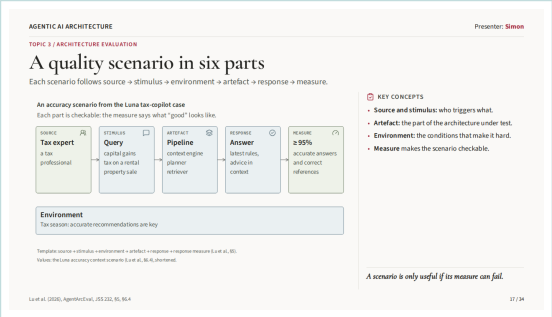
OPTIONAL PREPARATION — example or clarification

At design time, bring stakeholders, goals, architecture views and constraints; leave with prioritised scenarios, identified risks and proposed changes. At runtime, use traces, outcomes and feedback to detect broken assumptions, revise scenarios and repeat the analysis. Optional illustration: a tool API change causes failures, prompting interface validation and fallback review. This is an illustrative application, not a Luna result.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232, §4, Fig. 2

17 / 34 A quality scenario in six parts

Topic 3 / Architecture evaluation • Simon



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Walk the six parts left to right; stress that the measure can fail.
- Timing: about 85 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Each scenario follows source → stimulus → environment → artefact → response → measure.
- **Source and stimulus:** who triggers what.
- **Artefact:** the part of the architecture under test.
- **Environment:** the conditions that make it hard.
- **Measure** makes the scenario checkable.
- *A scenario is only useful if its measure can fail.*

SPOKEN SCRIPT — short paragraphs for delivery

- A quality scenario makes an architectural requirement concrete. It has six parts: source, stimulus, environment, artefact, response and response measure. This example is from Luna, the paper's tax-copilot case study.
- The source is a tax professional. The stimulus is a question about capital gains tax on a rental-property sale. The environment is tax season. The artefact is the affected pipeline: context, planning, execution, retrieval and generation, including the knowledge store. The response is context-specific advice supported by current regulations.
- The measure contains two targets: at least ninety-five percent relevant-response accuracy and ninety-five percent correct references. These are requirements, not results the paper reports achieving. Testing them would require a defined question set and reference judgements.
- The scenario directs our review to the whole pipeline: even a capable model cannot reliably compensate for missing or outdated retrieved evidence.

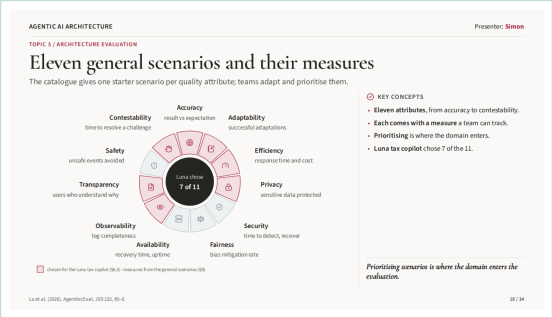
OPTIONAL PREPARATION — example or clarification

General scenario: a user requests a correct answer. Context scenario: a tax professional asks about a specified transaction during tax season, affecting named components, with explicit targets. Luna specifies two 95% requirements, not model confidence or measured success. For a real test, define reference answers, citation correctness, sample size and treatment of partial answers.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232, §5, §6.4

18 / 34 Eleven general scenarios and their measures

Topic 3 / Architecture evaluation • Simon



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Do not read all eleven; pick accuracy, safety and contestability, then reveal Luna's seven.
- Timing: about 65 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The catalogue gives one starter scenario per quality attribute; teams adapt and prioritise them.
- **Eleven attributes**, from accuracy to contestability.
- **Each comes with a measure** a team can track.
- **Prioritising** is where the domain enters.
- **Luna tax copilot** chose 7 of the 11.
- *Prioritising scenarios is where the domain enters the evaluation.*

SPOKEN SCRIPT — short paragraphs for delivery

- The catalogue gives eleven starting scenarios, one per quality attribute. A general scenario becomes a context scenario when a team supplies its own users, conditions, components and acceptance measures.
- Take accuracy: does the answer meet the user's expectation? Safety: are harmful actions avoided? Contestability: can someone challenge an outcome and receive meaningful review? Observability is different again: can an operator reconstruct the relevant execution from records?
- Luna's team selected seven attributes for its detailed analysis. That does not establish that the other four are irrelevant; it describes the scope of this case study. Prioritisation should reflect domain risk and stakeholder needs, with omissions made explicit.
- The case demonstrates a qualitative architecture review and resulting design changes. It does not prove that every numerical target was reached or that the method has been validated across all agent applications.

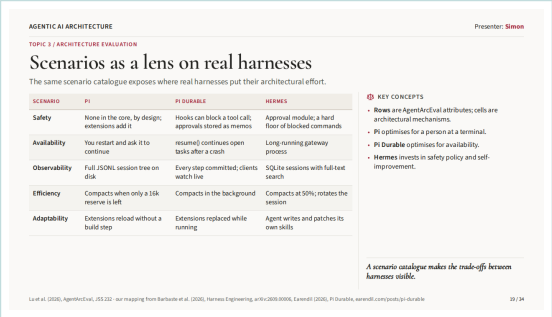
OPTIONAL PREPARATION — example or clarification

Observability: an operator reconstructs execution. Transparency: a user understands an outcome and its supporting sources. Contestability: the user challenges it and receives review. Luna requires contested cases to be resolved within 48 hours; that is a case-specific target. Safety and security can overlap, but accidental harmful action does not require a malicious attacker.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232, §5–6

19 / 34 Scenarios as a lens on real harnesses

Topic 3 / Architecture evaluation • Simon



CUES

- Static figure or table: no builds. If it is a paper figure, name the source aloud first.
- Do: Read the Safety row across all three columns, then one more row. Hand over to Luca.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The same scenario catalogue exposes where real harnesses put their architectural effort.
- **Rows** are AgentArcEval attributes; cells are architectural mechanisms.
- **Pi** optimises for a person at a terminal.
- **Pi Durable** optimises for availability.
- **Hermes** invests in safety policy and self-improvement.
- *A scenario catalogue makes the trade-offs between harnesses visible.*

SPOKEN SCRIPT — short paragraphs for delivery

- This comparison is our application of the scenario catalogue, not a result reported by AgentArcEval. Each cell names a mechanism to investigate, rather than a quality score.
- Take availability. Pi's terminal workflow relies on a person to continue after interruption. Pi Durable stores task checkpoints and resumes unfinished work. To evaluate that mechanism, interrupt a tool call, restart the process and check recovery time, task completion and whether an external action happened twice.
- For safety, examine where restrictions are enforced: Pi extensions, Pi Durable hooks, or Hermes's approval controls. A mechanism being present does not establish that every deployment is safe; its configuration and failure behaviour matter.
- The catalogue helps us connect a quality requirement to a design choice and then to evidence. Luca now examines what information those mechanisms place in the agent's context.

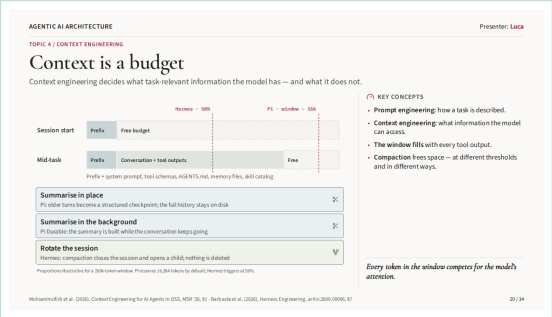
OPTIONAL PREPARATION — example or clarification

Optional test example: crash after a tool starts but before its result is recorded. On restart, inspect pending state and external effects. A checkpoint cannot alone prove whether an external action completed. Safe replay, deduplication and human review address different parts of this uncertainty. The table is our mapping of mechanisms, not a comparative benchmark.

Reading anchor: Lu et al. (2026), AgentArcEval, JSS 232
our mapping from Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, Earendil (2026), Pi Durable, earendil.com/posts/pi-durable

20 / 34 Context is a budget

Topic 4 / Context engineering • Luca



CUES

- Handover: **Luca** takes over here.
- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Point at the two threshold lines, then the three ways to free space.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Context engineering decides what task-relevant information the model has — and what it does not.
- **Prompt engineering:** how a task is described.
- **Context engineering:** what information the model can access.
- **The window fills** with every tool output.
- **Compaction** frees space — at different thresholds and in different ways.
- *Every token in the window competes for the model's attention.*

SPOKEN SCRIPT — short paragraphs for delivery

- Thank you, Simon. Prompt engineering is about how a task is described. Context engineering is about what information the model can access at all.
- The window is a budget. A fixed prefix — instructions, tool schemas, context files — sits at the start, and every tool output adds to the rest. Harnesses compact at different points: Hermes at half the window, Pi only when a reserve of about sixteen thousand tokens is left.
- And they free space in different ways. Pi summarises older turns but keeps the full history on disk. Pi Durable builds the summary in the background while the conversation continues. Hermes closes the session and opens a child session, so nothing is deleted.

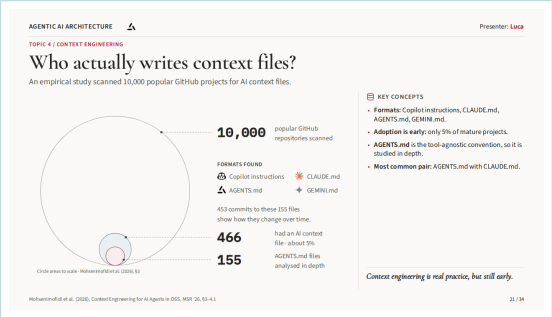
OPTIONAL PREPARATION — example or clarification

A token is a unit of model input or output; the context window has a finite capacity. Compaction replaces older material with a shorter representation. Optional illustration: keep the failing test and current plan while summarising a long search history. A preserved transcript on disk is not the same as details remaining visible to the next model call.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §1
Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §7

21 / 34 Who actually writes context files?

Topic 4 / Context engineering • Luca



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Reveal the circles from largest to smallest: 10,000, then 466, then 155.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- An empirical study scanned 10,000 popular GitHub projects for AI context files.
- **Formats:** Copilot instructions, CLAUDE.md, AGENTS.md, GEMINI.md.
- **Adoption is early:** only 5% of mature projects.
- **AGENTS.md** is the tool-agnostic convention, so it is studied in depth.
- **Most common pair:** AGENTS.md with CLAUDE.md.
- *Context engineering is real practice, but still early.*

SPOKEN SCRIPT — short paragraphs for delivery

- How common are these context files in practice? An empirical study scanned ten thousand popular GitHub projects. The circles are drawn to scale.
- Only four hundred and sixty-six — about five percent — had any AI context file. They come in several formats: Copilot instructions, CLAUDE.md, AGENTS.md and GEMINI.md. The most common pair is AGENTS.md with CLAUDE.md.
- AGENTS.md is the tool-agnostic convention, so the authors studied one hundred and fifty-five of those files in depth, plus four hundred and fifty-three commits to them. Context engineering is real practice, but still early.

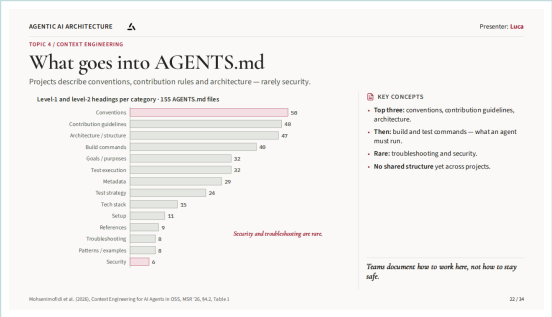
OPTIONAL PREPARATION — example or clarification

Keep the denominators distinct: 466 of 10,000 sampled repositories had any supported context-file format; 155 AGENTS.md files formed the detailed content sample. Absence of such a file does not establish absence of AI use. The findings describe this repository sample and observation period, not adoption across all software projects.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §3–4.1

22 / 34 What goes into AGENTS.md

Topic 4 / Context engineering • Luca



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Point at the top bar, then at the security bar at the bottom.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Projects describe conventions, contribution rules and architecture — rarely security.
- **Top three:** conventions, contribution guidelines, architecture.
- **Then:** build and test commands — what an agent must run.
- **Rare:** troubleshooting and security.
- **No shared structure** yet across projects.
- *Teams document how to work here, not how to stay safe.*

SPOKEN SCRIPT — short paragraphs for delivery

- What do people write in these files? The most frequent categories are conventions, contribution guidelines and architecture. Then build and test commands — the things an agent must actually run.
- Troubleshooting and security are rare. And there is no shared structure across projects yet.
- So these files more often document how to work in a repository than how to stay safe. That does not tell us whether separate security controls exist elsewhere.

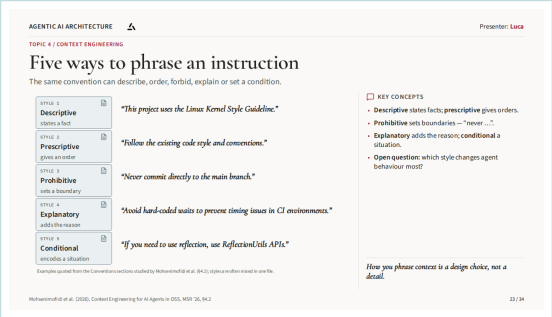
OPTIONAL PREPARATION — example or clarification

Optional illustration: specify the actual test command and where generated files belong, rather than saying only to follow best practices. Rare security text in these files is a documentation finding, not proof of missing access controls. Do not infer that a longer instruction file is more effective; the study describes practice rather than demonstrating causal improvement.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §4.2, Table 1

23 / 34 Five ways to phrase an instruction

Topic 4 / Context engineering • Luca



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Read two quotes aloud: one prescriptive, one prohibitive.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The same convention can describe, order, forbid, explain or set a condition.
- **Descriptive** states facts; **prescriptive** gives orders.
- **Prohibitive** sets boundaries — “never …”.
- **Explanatory** adds the reason; **conditional** a situation.
- **Open question:** which style changes agent behaviour most?
- *How you phrase context is a design choice, not a detail.*

SPOKEN SCRIPT — short paragraphs for delivery

- The same convention can be phrased in five styles. Descriptive: “This project uses the Linux Kernel Style Guideline.” Prescriptive: “Follow the existing code style.” Prohibitive: “Never commit directly to main.”
- Explanatory instructions add the reason, and conditional ones describe a situation: “If you need reflection, use these APIs.” Real files mix styles.
- An open research question is which style changes agent behaviour most. How you phrase context is a design choice, not a detail.

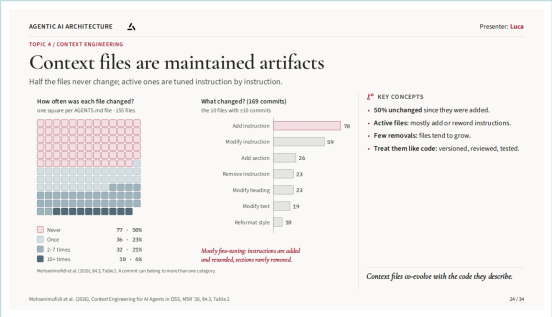
OPTIONAL PREPARATION — example or clarification

Optional illustration of one rule in two styles: descriptive, the build outputs are generated; prohibitive, do not edit the build outputs directly. Explanatory wording can add that rebuilding would overwrite changes. These are examples to explain categories, not evidence that one style reliably makes an agent comply.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR ’26, §4.2

24 / 34 Context files are maintained artifacts

Topic 4 / Context engineering • Luca



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Point at the crimson half of the squares first, then the right chart.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Half the files never change; active ones are tuned instruction by instruction.
- **50% unchanged** since they were added.
- **Active files:** mostly add or reword instructions.
- **Few removals:** files tend to grow.
- **Treat them like code:** versioned, reviewed, tested.
- *Context files co-evolve with the code they describe.*

SPOKEN SCRIPT — short paragraphs for delivery

- Do these files change after they are written? Each square is one file. Half of them, the crimson squares, never changed. A small group changed ten times or more.
- In those active files, most changes add or reword instructions. Sections are rarely removed, so the files tend to grow.
- That suggests treating context files like code: versioned, reviewed and tested, because they co-evolve with the code they describe.

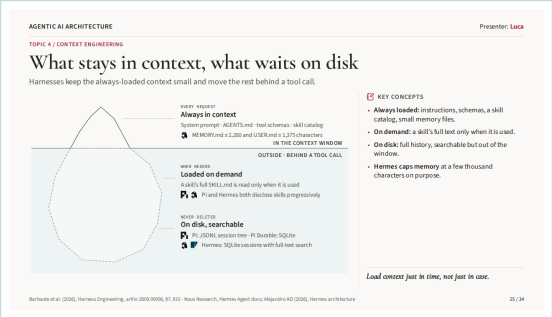
OPTIONAL PREPARATION — example or clarification

An unchanged file may remain valid, while a frequently changed file may still contain contradictions. Optional maintenance check: after the test command changes, update the instruction file and verify that the documented command runs. Commit counts describe editing activity, not instruction quality or whether an agent follows the file.

Reading anchor: Mohsenimofidi et al. (2026), Context Engineering for AI Agents in OSS, MSR '26, §4.3, Table 2

25 / 34 What stays in context, what waits on disk

Topic 4 / Context engineering • Luca



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Start above the waterline, then go deeper: on demand, then on disk.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Harnesses keep the always-loaded context small and move the rest behind a tool call.
- **Always loaded:** instructions, schemas, a skill catalog, small memory files.
- **On demand:** a skill's full text only when it is used.
- **On disk:** full history, searchable but out of the window.
- **Hermes caps memory** at a few thousand characters on purpose.
- *Load context just in time, not just in case.*

SPOKEN SCRIPT — short paragraphs for delivery

- Think of context as an iceberg. Only the tip sits in the context window: what every request carries: instructions, tool schemas, a skill catalog and small memory files. Hermes deliberately caps its memory files at a few thousand characters.
- Below the waterline is what loads on demand. Only a skill's name and description are always present; its full text is read when the skill is used. Pi and Hermes both work this way.
- Deepest down is everything on disk: the full history, searchable but outside the window. Pi keeps a session tree of JSON lines, Pi Durable an SQLite transcript, Hermes SQLite sessions with full-text search. Load context just in time, not just in case.

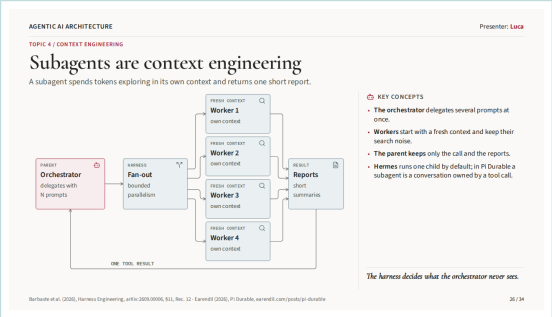
OPTIONAL PREPARATION — example or clarification

Optional illustration: keep a skill name and short description in context; load its full instructions when needed; search older session records only for relevant details. SQLite is an embedded database, and full-text search finds words in stored records. Keeping history on disk does not guarantee successful retrieval or faithful summarisation.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §7, §15
Nous Research, Hermes Agent docs; Alejandro AO (2026), Hermes architecture

26 / 34 Subagents are context engineering

Topic 4 / Context engineering • Luca



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Follow the fan-out to the workers and back to the single tool result. Then hand over to Matheus.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- A subagent spends tokens exploring in its own context and returns one short report.
- **The orchestrator** delegates several prompts at once.
- **Workers** start with a fresh context and keep their search noise.
- **The parent keeps** only the call and the reports.
- **Hermes** runs one child by default; in Pi Durable a subagent is a conversation owned by a tool call.
- *The harness decides what the orchestrator never sees.*

SPOKEN SCRIPT — short paragraphs for delivery

- Subagents are usually explained as parallelism. They are also context engineering.
- The orchestrator delegates several prompts at once. Each worker starts with a fresh context, spends its tokens exploring, and returns a short report. The parent keeps only the call and the reports, never the workers' search noise.
- Real systems keep this bounded. Hermes runs one child by default. In Pi Durable a subagent is simply a conversation owned by a tool call, so it survives a crash like everything else. Matheus now asks what happens when agents stop waiting for us.

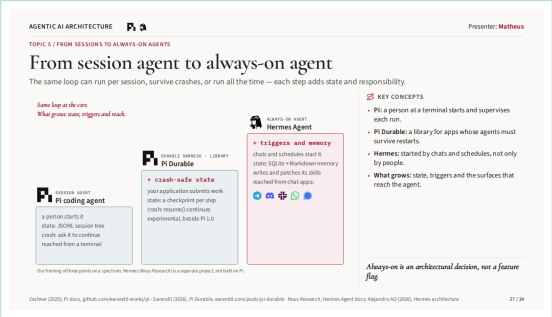
OPTIONAL PREPARATION — example or clarification

Optional illustration: one worker investigates a test failure and another reads an interface, each returning a short evidence-based report. The parent avoids carrying every search result but can lose important detail in summaries. Independent context is not necessarily isolation of files, permissions or external side effects; those need separate boundaries.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §11, Rec. 12
Earendil (2026), Pi Durable, earendil.com/posts/pi-durable

27 / 34 From session agent to always-on agent

Topic 5 / From sessions to always-on agents • Matheus



CUES

- Handover: **Matheus** takes over here.
- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Climb the stairs one step at a time; stress that this spectrum is our own synthesis.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The same loop can run per session, survive crashes, or run all the time — each step adds state and responsibility.
- **Pi:** a person at a terminal starts and supervises each run.
- **Pi Durable:** a library for apps whose agents must survive restarts.
- **Hermes:** started by chats and schedules, not only by people.
- **What grows:** state, triggers and the surfaces that reach the agent.
- *Always-on is an architectural decision, not a feature flag.*

SPOKEN SCRIPT — short paragraphs for delivery

- Thank you, Luca. The same loop can run in very different ways. This spectrum is our own synthesis, not a lineage the sources claim.
- On the first step, the Pi coding agent: a person starts each run at a terminal and supervises it; if it crashes, you ask it to continue. One step up, Pi Durable: a library from the same team, released beside Pi 1.0 on the first of October, for applications whose agents must survive restarts. On the top step, Hermes from Nous Research: started by chat messages and schedules, not only by people.
- Each step up adds state, triggers and the surfaces that can reach the agent. Always-on is an architectural decision, not a feature flag.

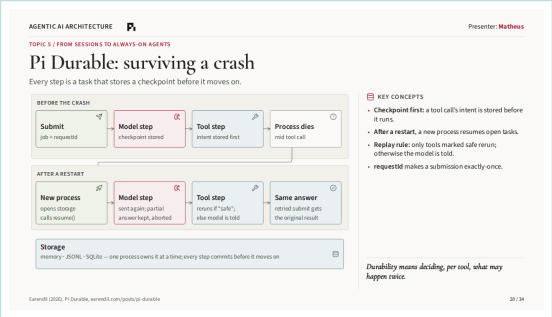
OPTIONAL PREPARATION — example or clarification

Distinguish durable execution from autonomous triggering. A runtime can survive a crash while still waiting for a person to submit work. A scheduler can launch work automatically without robust recovery. Optional illustration: a nightly report needs both a time trigger and a policy for interrupted or duplicate runs.

Reading anchor: Zechner (2025); Pi docs, github.com/earendil-works/pi
Earendil (2026), Pi Durable, earendil.com/posts/pi-durable
Nous Research, Hermes Agent docs; Alejandro AO (2026), Hermes architecture

28 / 34 Pi Durable: surviving a crash

Topic 5 / From sessions to always-on agents • Matheus



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Upper lane, then the storage bar, then the lower lane after the restart.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Every step is a task that stores a checkpoint before it moves on.
- **Checkpoint first:** a tool call's intent is stored before it runs.
- **After a restart**, a new process resumes open tasks.
- **Replay rule:** only tools marked safe rerun; otherwise the model is told.
- **requestId** makes a submission exactly-once.
- *Durability means deciding, per tool, what may happen twice.*

SPOKEN SCRIPT — short paragraphs for delivery

- How does an agent survive a crash? In Pi Durable every step of a run is a task that stores a checkpoint before it moves on.
- Upper lane: a job is submitted with a request ID. The model step is checkpointed. Before a tool runs, its intent is stored. Then the process dies in the middle of the tool call.
- Lower lane: a new process opens the same storage and calls resume. A cut-off model request is sent again; the partial answer stays, marked as aborted. A tool reruns only if it is marked safe to replay; otherwise the model is told it was interrupted. A client retry with the same request ID finds the original submission. This deduplicates submissions; it does not make every external side effect exactly-once. Durability means deciding, per tool, what may happen twice.

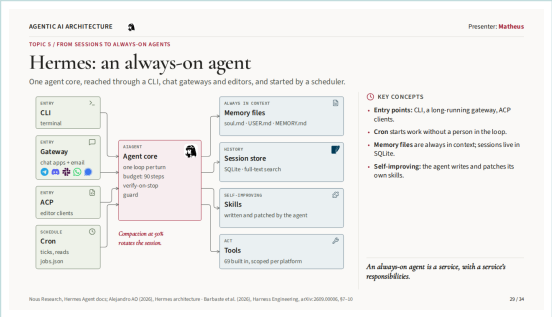
OPTIONAL PREPARATION — example or clarification

A checkpoint stores progress; replay reruns an interrupted step; idempotency means repeating an operation has the same intended effect as doing it once. Optional illustration: retrying a read is usually simpler than retrying a payment. A request ID deduplicates submissions, but an external action needs its own duplicate-prevention or reconciliation mechanism.

Reading anchor: Earendil (2026), Pi Durable, earendil.com/posts/pi-durable

29 / 34 Hermes: an always-on agent

Topic 5 / From sessions to always-on agents • Matheus



CUES

- Build: press Right 3 times to reveal the complete figure; once more to advance.
- Do: Entry points first, then cron, then the four boxes on the right.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- One agent core, reached through a CLI, chat gateways and editors, and started by a scheduler.
- **Entry points:** CLI, a long-running gateway, ACP clients.
- **Cron** starts work without a person in the loop.
- **Memory files** are always in context; sessions live in SQLite.
- **Self-improving:** the agent writes and patches its own skills.
- *An always-on agent is a service, with a service's responsibilities.*

SPOKEN SCRIPT — short paragraphs for delivery

- Hermes shows the full always-on shape. Three entry points feed one agent core: the command line, a long-running gateway connected to chat apps like Telegram and Slack, and editor clients over the Agent Client Protocol, or ACP. ACP connects an editor to an agent; MCP connects an agent to tools and resources.
- A scheduler can start work without anyone typing: it reads a jobs file and runs tasks on a timer. When the context reaches half the window, compaction closes the session and starts a child session.
- On the right: memory files always in context, an SQLite session store with full-text search, skills that the agent writes and patches itself, and sixty-nine tools scoped per platform. An always-on agent is a service, with a service's responsibilities.

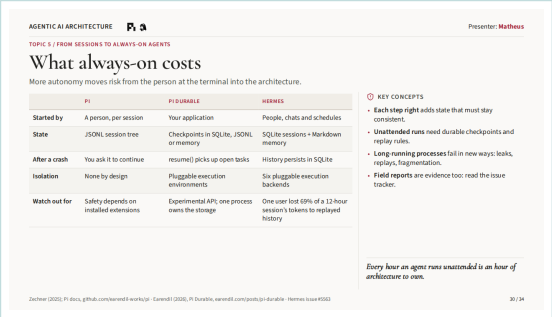
OPTIONAL PREPARATION — example or clarification

ACP means Agent Client Protocol: it connects editors and agent clients to an agent. MCP means Model Context Protocol: it connects agents to tools and resources. A gateway receives messages, while a scheduler initiates timed work. Optional illustration: both a chat request and a scheduled report can reach the same agent core with different permissions.

Reading anchor: Nous Research, Hermes Agent docs; Alejandro AO (2026), Hermes architecture
Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §7–10

30 / 34 What always-on costs

Topic 5 / From sessions to always-on agents • Matheus



CUES

- Static figure or table: no builds. If it is a paper figure, name the source aloud first.
- Do: Read the last row across all three columns.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- More autonomy moves risk from the person at the terminal into the architecture.
- **Each step right** adds state that must stay consistent.
- **Unattended runs** need durable checkpoints and replay rules.
- **Long-running processes** fail in new ways: leaks, replays, fragmentation.
- **Field reports** are evidence too: read the issue tracker.
- *Every hour an agent runs unattended is an hour of architecture to own.*

SPOKEN SCRIPT — short paragraphs for delivery

- What does that autonomy cost? Read the table left to right. Who starts the work, where the state lives, what happens after a crash, and how the agent is isolated.
- The last row is the one to remember. With Pi, safety depends on the extensions you install. Pi Durable is experimental, and only one process may own the storage. For Hermes, one user documented that sixty-nine percent of a twelve-hour session's tokens went to replaying history after the session split. That is a single field report, not a benchmark, but it shows a new class of failure.
- Issue trackers are evidence too. Every hour an agent runs unattended is an hour of architecture to own.

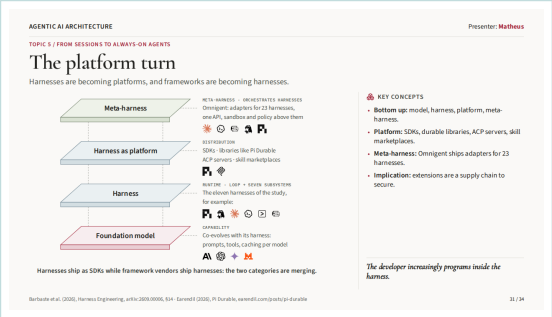
OPTIONAL PREPARATION — example or clarification

The 69% history-replay figure is one user report, not a typical rate for Hermes. Optional operational example: measure tokens per completed job, recovery attempts and unresolved failures over time. More stored history can improve continuity while increasing replay costs. Always-on operation also needs a clear owner for failures, credentials and pending approvals.

Reading anchor: Zechner (2025); Pi docs, github.com/earendil-works/pi
Earendil (2026), Pi Durable, earendil.com/posts/pi-durable
Hermes issue #5563

31 / 34 The platform turn

Topic 5 / From sessions to always-on agents • Matheus



CUES

- Build: press Right 2 times to reveal the complete figure; once more to advance.
- Do: Read the plates bottom up; name one logo per layer.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- Harnesses are becoming platforms, and frameworks are becoming harnesses.
- **Bottom up:** model, harness, platform, meta-harness.
- **Platform:** SDKs, durable libraries, ACP servers, skill marketplaces.
- **Meta-harness:** Omnigent ships adapters for 23 harnesses.
- **Implication:** extensions are a supply chain to secure.
- *The developer increasingly programs inside the harness.*

SPOKEN SCRIPT — short paragraphs for delivery

- Zoom out. At the bottom is the foundation model. Above it, the harness. Above that, the harness becomes a platform: software development kits, or SDKs, libraries like Pi Durable, editor-facing ACP servers, and skill marketplaces. At the top, a meta-harness: Omnigent, open-sourced by Databricks, ships adapters for twenty-three harnesses and puts one API, one sandbox and one policy layer above them.
- Harnesses ship as SDKs while framework vendors ship harnesses; the two categories are merging.
- One implication for architecture: extensions and skills become a supply chain that we have to secure.

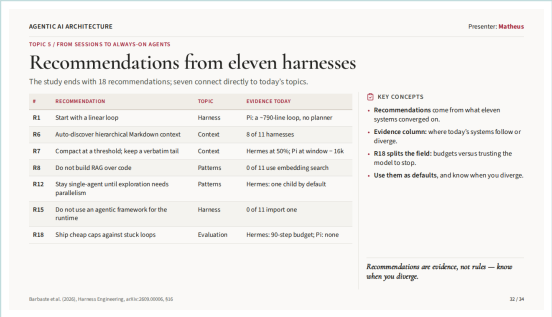
OPTIONAL PREPARATION — example or clarification

An SDK is a software development kit for integrating a system into another application. An adapter translates interfaces. Optional illustration: a common API can launch two different harnesses, but their permission and recovery semantics may still differ. A shared interface alone does not establish equivalent guarantees. Extensions introduce dependencies whose privileges and updates must be reviewed.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §14
Earendil (2026), Pi Durable, earendil.com/posts/pi-durable

32 / 34 Recommendations from eleven harnesses

Topic 5 / From sessions to always-on agents • Matheus



CUES

- Static figure or table: no builds. If it is a paper figure, name the source aloud first.
- Do: Pick R1, R8 and R18; ask the room which one they would break.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The study ends with 18 recommendations; seven connect directly to today’s topics.
- **Recommendations** come from what eleven systems converged on.
- **Evidence column:** where today’s systems follow or diverge.
- **R18 splits the field:** budgets versus trusting the model to stop.
- **Use them as defaults**, and know when you diverge.
- *Recommendations are evidence, not rules — know when you diverge.*

SPOKEN SCRIPT — short paragraphs for delivery

- The harness study ends with eighteen recommendations. Seven connect to today’s topics, and the last column shows the evidence behind them.
- Start with a linear loop: Pi’s loop is about seven hundred and ninety lines with no planner. Don’t build retrieval over code: none of the eleven does. Don’t use an agentic framework for the runtime: none imports one.
- Recommendation eighteen splits the field: ship cheap caps against stuck loops. Hermes has a ninety-step budget; Pi has none. Use these as defaults, and know when you diverge.

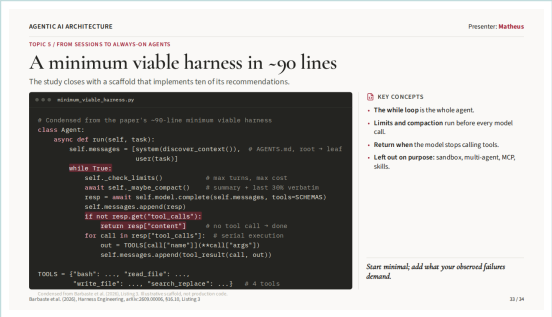
OPTIONAL PREPARATION — example or clarification

These recommendations are conclusions drawn from the studied corpus, not universal laws. Optional illustration: start with a simple loop, then add a cap after observing repeated retries. Existing retrieval infrastructure or domain requirements may justify a different choice. Explain the evidence and the workload before treating a recommendation as a default.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §16

33 / 34 A minimum viable harness in 90 lines

Topic 5 / From sessions to always-on agents • Matheus



CUES

- Static figure or table: no builds. If it is a paper figure, name the source aloud first.
- Do: Point at the while loop, then at the return line.
- Timing: about 70 seconds spoken; preparation is optional.

KEY CONCEPTS (as shown on the slide)

- The study closes with a scaffold that implements ten of its recommendations.
- **The while loop** is the whole agent.
- **Limits and compaction** run before every model call.
- **Return when** the model stops calling tools.
- **Left out on purpose:** sandbox, multi-agent, MCP, skills.
- *Start minimal; add what your observed failures demand.*

SPOKEN SCRIPT — short paragraphs for delivery

- The paper closes with a scaffold of about ninety lines. We condensed it here.
- Look at the while loop: that is the whole agent. Before each model call it checks limits and compacts if needed. It calls the model, appends the response, and returns when there are no more tool calls. Otherwise it runs each tool and appends the result.
- Sandboxing, multiple agents, MCP and skills are left out on purpose. Start minimal, and add what your observed failures demand.

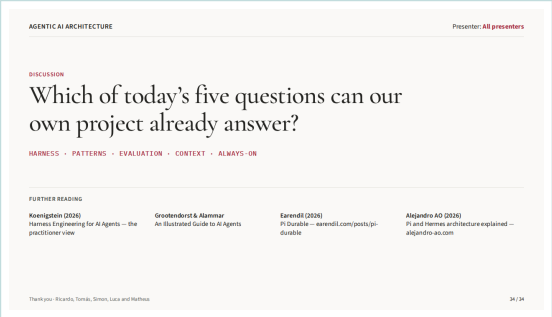
OPTIONAL PREPARATION — example or clarification

The snippet illustrates control flow, not a production-ready runtime. Optional failure example: a tool throws an exception; the implementation must decide what to record, whether to retry, and whether to stop. A model response without tool calls ends the loop, but task success requires separate evidence. Permissions and secret handling remain implementation responsibilities.

Reading anchor: Barbaste et al. (2026), Harness Engineering, arXiv:2609.00006, §16.10, Listing 3

34 / 34 Discussion

Discussion • All presenters



CUES

- Handover: **All presenters** takes over here.
- Do: Everyone comes forward. Matheus moderates.
- Timing: about 60 seconds spoken; preparation is optional.

SPOKEN SCRIPT — short paragraphs for delivery

- That brings us to the discussion. We asked five questions today: what a harness is, which patterns describe an agent, how to evaluate its architecture, how to engineer its context, and what changes when it runs all the time.
- Which of these questions can your own project already answer? And which would you need to answer before letting an agent run unattended?
- The further reading is on the slide. Thank you.

OPTIONAL PREPARATION — example or clarification

Optional prompt if discussion stalls: choose one agent task, name its highest-risk quality attribute, and write a concrete trigger, expected response and measure. Then identify the component or control that would support it. Ask what evidence is missing before the group would let that task run unattended.